

КУРСОВА РОБОТА

з дисципліни

ОК 31 Професійна практика програмної інженерії

на тему: “Розробка back-end частини на тему «Рейтингова таблиця водіїв симрейсингу»”

Здобувача вищої освіти

четвертого року навчання, групи ПЗ-41
освітньої програми

Інженерія програмного забезпечення
спеціальності

121 Інженерія програмного забезпечення
першого (бакалаврського) рівня вищої
освіти

Юрій ГАРПУЛЬ

Керівник: професор кафедри
інформаційних технологій, доктор
технічних наук

Микола КУЗЬ

Національна шкала: _____

Університетська шкала: _____

Оцінка ECTS: _____

Члени комісії: _____
(підпис)

(підпис)

Сергій ІЩЕРЯКОВ

Борис НЕЗАМАЙ

Завдання на курсову роботу

Гарпуль Юрій Михайлович

Кафедра інформаційних технологій

Дисципліна Професійна практика програмної інженерії

Спеціальність 121 Інженерія програмного забезпечення

Освітня програма Інженерія програмного забезпечення

Рік навчання 4 Група 1 Семестр 7

1.Тема проєкту Розробка back-end частини на тему «Рейтингова таблиця водіїв симрейсингу»

2.Перелік питань, які підлягають розробці: Проєктування архітектури серверної частини системи. Вибір технологій та інструментів для реалізації back-end частини. Розробка структури бази даних та моделей зберігання телеметричних даних. Реалізація механізмів завантаження, обробки та аналізу телеметрії заїздів.

3.Дата видачі завдання 03.10.2025

4.Термін подачі до захисту 19.12.2025

5.Здобувач вищої освіти: Юрій ГАРПУЛЬ

6.Керівник: Микола КУЗЬ

КАЛЕНДАРНИЙ ПЛАН

написання курсової роботи (проєкту) здобувачами вищої освіти спеціальності 121 (F2)
Інженерія програмного забезпечення у першому семестрі

№	Назва етапів підготовки курсового проєкту	Термін виконання	Примітки
1.	Затвердження теми роботи (проєкту) на засіданні випускової кафедри.	03.10.2025	
2.	Складання та затвердження індивідуального завдання та формування плану роботи (проєкту).	07.10.2025	
3.	Обґрунтування актуальності, формулювання мети, завдання, предмету та об'єкту дослідження роботи (проєкту). Підготовка вступу.	12.10.2025	
4.	Опрацювання джерел з теми роботи (проєкту). Підготовка I розділу.	18.10.2025	
5.	Виправлення зауважень. Підготовка II розділу.	30.10.2025	
6.	Виправлення зауважень. Підготовка III розділу.	15.11.2025	
7.	Виправлення зауважень. Підготовка висновків.	22.11.2025	
8.	Оформлення роботи (проєкту) згідно вимог.	30.11.2025	
9.	Виправлення зауважень керівника.	15.12.2025	
10.	Надсилання електронної версії роботи (проєкту) на перевірку на плагіат та подача друкованої роботи (проєкту) із усіма необхідними матеріалами на випускову кафедру.	19.12.2025	
11.	Захист роботи (проєкту).	22.01.2025	

Здобувач вищої
освіти

(підпис)

Юрій ГАРПУЛЬ
(ПІБ)

Керівник роботи

(підпис)

Микола КУЗЬ
(ПІБ)

РЕФЕРАТ

Пояснювальна записка : 44 сторінки, 11 рисунків, 9 джерел, 1 додаток

КЛЮЧОВІ СЛОВА: back-end, NestJS, Node.js, REST API, MongoDB, Mongoose, телеметрія, симрейсинг, MoTeC, обробка даних, рейтингова таблиця, вебзастосунок.

Мета проєкту полягає в розробці програмного забезпечення для автоматизованої обробки телеметричних даних, отриманих із файлів автосимулятора, та збереження результатів аналізу у базі даних для формування рейтингової таблиці водіїв симрейсингу.

Стислий опис тексту: Backend API на NestJS для аналізу телеметрії симуляторів. Забезпечує парсинг файлів MoTeC (.ld/.ldx), вилучення детальної інформації про сесію (пілот, автомобіль, траса, час кіл, швидкості, гальмування, прискорення), а також додаткових телеметричних параметрів. API дозволяє зберігати та структуровано організовувати результати у MongoDB для подальшого аналізу, візуалізації та порівняння сесій.

ABSTRACT

Explanatory note: 44 pages, 11 figures, 9 sources, 1 appendix

KEYWORDS: back-end, NestJS, Node.js, REST API, MongoDB, Mongoose, telemetry, sim racing, MoTeC, data processing, ranking table, web application.

Project goal: The goal of the project is to develop software for automated processing of telemetry data obtained from racing simulator files and storing the analysis results in a database to generate a ranking table of sim racing drivers.

Brief description of the text: Backend API on NestJS for analyzing simulator telemetry. It provides parsing of MoTeC files (.ld/.ldx), extraction of detailed session information (driver, car, track, lap times, speeds, braking, acceleration), as well as additional telemetry parameters. The API allows storing and organizing the results in MongoDB for further analysis, visualization, and session comparison.

ЗМІСТ		
ВСТУП		6
1 ТЕОРЕТИЧНІ ВІДОМОСТІ ТА АНАЛІЗ ПРИНЦИПІВ ПОБУДОВИ ВЕБ-ОРІЄНТОВАНИХ СИСТЕМ ОБРОБКИ ДАНИХ		7
1.1	Архітектурні принципи побудови сучасних веб-систем	7
1.2	Аналіз методів та засобів обробки телеметричних даних	8
1.3	Обґрунтування вибору технологічного стеку	10
2 ПРОЄКТУВАННЯ АРХІТЕКТУРИ ТА КОМПОНЕНТІВ СИСТЕМИ АНАЛІЗУ ТЕЛЕМЕТРІЇ		12
2.1	Розробка архітектури серверного додатку	12
2.2	Проектування бази даних та схеми зберігання рейтингів	14
2.3	Проектування алгоритмів серверної логіки	17
2.4	Проектування архітектури REST API	18
3 ПРОГРАМНА РЕАЛІЗАЦІЯ СИСТЕМИ		20
3.1	Реалізація підсистеми доступу до даних	20
3.2	Програмна реалізація обробки телеметрії	22
3.3	Реалізація REST API інтерфейсів	23
3.4	Тестування та валідація працездатності системи	25
ВИСНОВКИ		28
ПЕРЕЛІК ПОСИЛАНЬ НА ДЖЕРЕЛА		29
ДОДАТОК А		30

					КП.ПЗ-4121.ПЗ										
Зм.	Арк.	№ докум.	Підпис	Дата											
Розробив		Ю. ГАРПУЛЬ			Розробка back-end частини на тему «Рейтингова таблиця водіїв симрейсингу»					Літ		Аркуш		Аркуші	
Перевірив		М. КУЗЬ								Н		6		31	
Н. Контр.		М. КУЗЬ								КНУВС ПЗ-41					
Затвердив		Б. НЕЗАМАЙ													

ВСТУП

Сучасний автосимуляторний спорт (сїмрейсінг) вже давно вийшов за межі звичайної розваги, перетворившись на повноцінну кіберспортивну дисципліну та інструмент для тренування професійних пілотів. Ключовим елементом підвищення майстерності в автоспорті є робота з даними телеметрії — детальним записом параметрів автомобіля та дій пілота під час заїзду.

Проте, існуючі професійні інструменти для аналізу (наприклад, MoTeC i2) є складними десктопними програмами, прив'язаними до локального робочого місця. Це створює труднощі при обміні даними між учасниками команди, тренерами та пілотами. Існує нагальна потреба у створенні веб-орієнтованих систем, які дозволяють централізовано зберігати, обробляти та надавати зручний доступ до результатів заїздів через браузер. Розробка такої системи з використанням сучасних технологій, таких як Node.js та хмарних баз даних, є актуальною задачею в контексті розвитку Industry 4.0 та SaaS-рішень.

Метою курсової роботи є проєктування та розробка веб-сервісу для автоматизованого збору, обробки та зберігання файлів телеметрії гоночних симуляторів.

					КП.ІПЗ-4121.ІЗ	Арк.
						7
Зм.	Арк.	№ докум.	Підпис	Дата		

1 ТЕОРЕТИЧНІ ВІДОМОСТІ ТА АНАЛІЗ ПРИНЦИПІВ ПОБУДОВИ ВЕБ-ОРІЄНТОВАНИХ СИСТЕМ ОБРОБКИ ДАНИХ

1.1 Архітектурні принципи побудови сучасних веб-систем

Фундаментом проєктування сучасних програмних продуктів є клієнт-серверна архітектура, яка передбачає чіткий, логічний та фізичний розподіл системи на дві незалежні складові. У цій моделі клієнтська частина, реалізована за допомогою сучасних бібліотек на кшталт React, виступає не просто засобом перегляду сторінок, а повноцінним програмним забезпеченням, що виконується безпосередньо у браузері користувача і бере на себе відповідальність за побудову інтерфейсу та взаємодію з людиною. Серверна ж частина, побудована на платформі Node.js, виконує роль централізованого обчислювального вузла, який обробляє бізнес-логіку, здійснює валідацію вхідних даних та забезпечує взаємодію з базою даних, залишаючись при цьому повністю відокремленим від візуальної складової. Така сегрегація дозволяє розробляти та масштабувати фронтенд і бекенд незалежно один від одного, об'єднуючи їх лише узгодженим інтерфейсом обміну інформацією.

Комунікація між розподіленими частинами системи відбувається на основі протоколу HTTP, а структура цієї взаємодії найчастіше підпорядковується архітектурному стилю REST API. Ключовою особливістю даного підходу є принцип Stateless, тобто відсутність збереження стану сесії на стороні сервера між різними запитами. Це означає, що сервер не запам'ятовує контекст попередніх дій користувача, тому кожен окремий запит від клієнта повинен бути самодостатнім і містити всю необхідну інформацію для його ідентифікації та коректної обробки. Сама передача даних базується на чіткій семантиці HTTP-методів, які визначають намір клієнта: метод GET використовується виключно для безпечного отримання ресурсів без внесення змін у систему, тоді як метод POST слугує для надсилання нових даних або ініціювання складних операцій обробки на стороні сервера.

					КП.ІІЗ-4121.ІЗ	Арк.
						8
Зм.	Арк.	№ докум.	Підпис	Дата		

Впровадження REST-архітектури та потужних клієнтських фреймворків уможливило перехід до концепції однострінкових додатків (Single Page Application або SPA), що стало стандартом сучасної веб-розробки. На відміну від традиційних багатострінкових сайтів, які при кожному переході користувача змушували браузер повністю перезавантажувати сторінку та заново отримувати всю розмітку, технологія SPA завантажує основний каркас додатку лише один раз під час першого візиту. Уся подальша взаємодія відбувається шляхом асинхронних фонових запитів до API, у відповідь на які сервер надсилає лише "сирі" дані, зазвичай у форматі JSON. Це дозволяє клієнтському додатку динамічно перемальовувати лише необхідні частини інтерфейсу, забезпечуючи швидку реакцію системи та плавний користувацький досвід, максимально наближений до роботи звичайних десктопних програм.

1.2 Аналіз методів та засобів обробки телеметричних даних

Ключовою особливістю даного проєкту є специфічна природа даних, з якими взаємодіє система. На відміну від класичних інформаційних ресурсів, де переважають статичні записи, робота з телеметрією автосимуляторів передбачає обробку складних часових рядів (Time Series Data). Кожен набір даних представляє собою хронологічно впорядковану послідовність вимірювань фізичних показників автомобіля. Критичним фактором тут виступає частота дискретизації — швидкість, з якою фіксуються параметри під час заїзду. Для забезпечення достатньої точності аналізу фізики руху та поведінки пілота, запис даних відбувається з високою частотою, яка зазвичай становить 60 Гц (60 разів на секунду) або більше. Це генерує щільний потік інформації, де навіть коротка сесія створює масиви, що складаються з сотень тисяч точок даних для кожного окремого параметру, такого як швидкість, положення керма чи температура шин.

Така висока деталізація породжує низку технічних викликів, пов'язаних із зберіганням та обробкою інформації. Для оптимізації використання дискового

					КП.ІІЗ-4121.ІЗ	Арк.
						9
Зм.	Арк.	№ докум.	Підпис	Дата		

простору та зменшення навантаження на систему введення-виведення, вихідні файли телеметрії (наприклад, у форматі Motec) зберігаються не у текстовому, а у бінарному (двійковому) вигляді. Хоча це ефективно з точки зору компресії, така структура є "непрозорою" для стандартних веб-інструментів та браузерів. Тому невід'ємним етапом роботи системи стає процес парсингу або десеріалізації — алгоритмічної трансформації "сирого" потоку байтів у структурований об'єктний вигляд, придатний для програмної обробки та візуалізації. Цей процес вимагає значних обчислювальних ресурсів і повинен бути максимально оптимізований, щоб мінімізувати час очікування користувача.

Вибір стратегії зберігання розпарсених даних є фундаментом архітектури бекенду, і тут виникає необхідність вибору між різними типами СУБД:

Реляційні бази даних (SQL): Традиційні табличні рішення (на кшталт PostgreSQL чи MySQL) демонструють низьку ефективність у роботі з телеметрією. Їхня вимога до жорсткої, заздалегідь визначеної схеми погано узгоджується з динамічною природою даних симуляторів, де набір каналів може змінюватися від машини до машини. Крім того, спроба нормалізувати такі дані призводить до створення надмірно великих таблиць, що ускладнює масштабування та сповільнює вибірку.

Документо-орієнтовані бази (NoSQL): Найбільш доцільним підходом є використання NoSQL рішень, таких як MongoDB. Їхня архітектура дозволяє зберігати дані у форматі JSON-подібних документів (BSON), що ідеально відображає ієрархічну структуру телеметричних об'єктів. Такий підхід забезпечує гнучкість схеми (schema-less), дозволяючи зберігати цілі заїзди або їх фрагменти як єдині документи, що значно прискорює операції читання та запису при побудові аналітичних графіків.

					КП.ІІЗ-4121.ІЗ	Арк.
						10
Зм.	Арк.	№ докум.	Підпис	Дата		

1.3 Обґрунтування вибору технологічного стеку

Проектування архітектури програмного комплексу базується на принципі "найкращий інструмент для конкретної задачі" (best-of-breed). Оскільки система повинна одночасно обробляти високошвидкісні мережеві запити та виконувати ресурсомісткі математичні обчислення над бінарними файлами, було прийнято рішення відмовитися від монолітної архітектури на одній мові на користь гібридного підходу.

Серверна платформа Node.js та фреймворк NestJS. Вибір Node.js як середовища виконання зумовлений його унікальною асинхронною моделлю, побудованою навколо циклу подій (Event Loop). У контексті веб-системи, яка приймає файли телеметрії, більшість операцій є операціями введення-виведення (I/O) — це очікування мережевих пакетів, запис файлів на диск та звернення до бази даних. Традиційні багатопотокові сервери виділяють окремий потік пам'яті на кожного клієнта, що швидко вичерпує ресурси при пікових навантаженнях. Node.js, навпаки, обробляє тисячі конкурентних з'єднань в одному потоці, не блокуючи його під час очікування I/O, що забезпечує високу пропускну здатність API. Для структурування коду використовується NestJS — прогресивний фреймворк, який привносить у світ JavaScript суворі архітектурні патерни, такі як впровадження залежностей (Dependency Injection) та модульність. Це дозволяє створювати надійну, масштабовану серверну архітектуру, яка легко піддається тестуванню та підтримці, що є критичним для довготривалого розвитку проєкту.

Гібридна архітектура обробки даних (Node.js + Python). Однією з ключових інновацій у проєкті є розділення відповідальності між двома мовами програмування. Node.js ідеально підходить для маршрутизації та мережевої взаємодії, проте він менш ефективний у важких обчислювальних задачах (CPU-bound tasks), до яких належить парсинг бінарних форматів Motec. Виконання таких операцій у головному потоці Node.js призвело б до блокування всього серверу та зупинки обробки запитів інших користувачів.

					КП.ІІЗ-4121.ІЗ	Арк.
						10
Зм.	Арк.	№ докум.	Підпис	Дата		

Вирішенням цієї проблеми стала реалізація мікросервісного підходу: Node.js діє як оркестратор, який приймає файл і запускає ізольований дочірній процес (child process) на Python. Python, у свою чергу, використовує оптимізовані бібліотеки (наприклад, Pandas або спеціалізовані парсери struct), щоб швидко розібрати бінарний код, виконати математичні перетворення та повернути структурований результат. Такий симбіоз поєднує реактивність веб-сервера зі швидкістю наукових обчислень.

					КП.ІІЗ-4121.ІЗ	Арк.
						12
Зм.	Арк.	№ докум.	Підпис	Дата		

2 ПРОЄКТУВАННЯ АРХІТЕКТУРИ ТА КОМПОНЕНТІВ СИСТЕМИ АНАЛІЗУ ТЕЛЕМЕТРІЇ

2.1 Розробка архітектури серверного додатку

Фундаментальна організація серверної частини (Backend) базується на принципах модульної архітектури (Modular Architecture), реалізація якої забезпечується екосистемою фреймворку NestJS. Такий підхід передбачає стратегічну декомпозицію складної системи на набір слабо зв'язаних, автономних функціональних блоків — модулів. Кожен модуль виступає контейнером для певної частини предметної області (Domain), наприклад, модуля автентифікації, модуля управління перегонами або аналітичного ядра, що дозволяє інкапсулювати логіку та значно спростити підтримку коду при масштабуванні проєкту.

Архітектурний каркас додатку формується з наступних ключових компонентів:

- Контролери (Controllers): Точки входу для зовнішніх запитів, що відповідають за маршрутизацію трафіку.
- Сервіси (Services): Провайдери, що містять основну обчислювальну логіку та алгоритми.
- Модуль обробки даних (Data Processing Module): Ізольоване середовище для виконання "важких" операцій парсингу.
- Рівень доступу до даних (Data Access Layer): Абстракція над базою даних для виконання CRUD-операцій.

Життєвий цикл обробки запиту в системі реалізовано через багат шаровий конвеєр (pipeline). Перш ніж запит потрапить до контролера, він проходить крізь систему вартів (Guards) та перехоплювачів (Interceptors). Це дозволяє винести аспекти безпеки, такі як перевірка JWT-токенів та авторизація ролей (RBAC), за межі бізнес-логіки. Після успішної авторизації в роботу вступають валідаційні труби (Validation Pipes).

					КП.ІІЗ-4121.ІЗ	Арк.
						13
Зм.	Арк.	№ докум.	Підпис	Дата		

Вони автоматично перевіряють вхідні дані на відповідність схемам DTO (Data Transfer Objects), гарантуючи, що в систему не потрапить некоректна або шкідлива інформація. Лише після цього "чисті" дані передаються методам контролера, який делегує виконання задачі відповідним сервісам.

Критично важливим аспектом архітектури є використання механізму ін'єкції залежностей (Dependency Injection, DI), вбудованого в ядро NestJS. Завдяки контейнеру інверсії керування (IoC), класи не створюють екземпляри своїх залежностей самостійно, а отримують їх ззовні. Це забезпечує високу гнучкість системи: наприклад, сервіс логування або базу даних можна легко замінити на інші реалізації без необхідності переписувати код контролерів. Також це значно спрощує модульне тестування (Unit Testing), дозволяючи підміняти реальні сервіси на тестові заглушки (mocks).

Окремої уваги заслуговує технічна реалізація модуля обробки телеметрії. Оскільки Node.js є однопоточковим середовищем, виконання інтенсивних математичних операцій (CPU-bound) безпосередньо в основному потоці призвело б до повного блокування Event Loop і недоступності сервера для інших користувачів. Архітектурне вирішення цієї проблеми полягає у використанні патерну Worker Processes. Node.js виступає в ролі менеджера, який через модуль `child_process` породжує незалежні екземпляри інтерпретатора Python. Комунікація між ними відбувається через стандартні потоки введення-виведення (stdin/stdout) або IPC-канали: Node.js передає буфер бінарних даних, а Python-скрипт, використовуючи бібліотеку `Pandas`, проводить парсинг і повертає структурований JSON. Такий асинхронний підхід дозволяє серверу залишатися чуйним навіть під час обробки гігабайтів телеметричних даних.

Взаємодія зі сховищем даних винесена на рівень Data Access Layer (DAL) з використанням ODM `Mongoose`. Це дозволяє працювати з документами MongoDB як з типізованими об'єктами, використовуючи можливості TypeScript. Архітектура передбачає створення індексів для полів, за якими найчастіше відбувається пошук (наприклад, ID пілота або дата заїзду), що забезпечує миттєву вибірку даних навіть при мільйонах записів у колекції.

					КП.ІІЗ-4121.ІЗ	Арк.
						14
Зм.	Арк.	№ докум.	Підпис	Дата		

2.2 Проектування бази даних та схеми зберігання рейтингів

Проектування схеми бази даних для системи телеметрії вимагає відходу від традиційної нормалізації, притаманної реляційним СУБД, на користь гнучкої документо-орієнтованої моделі. Для забезпечення ефективного ранжування пілотів та швидкого доступу до аналітичних даних було розроблено структуру, що базується на двох ключових сутностях, взаємодія між якими забезпечує цілісність та продуктивність системи.

Центральною ланкою механізму безпеки та персоналізації виступає колекція профілів користувачів, структура якої оптимізована для забезпечення швидкої автентифікації та підтримки цілісності глобального рейтингу. Фундаментом кожного запису слугує автоматично згенерований ідентифікатор `_id`, що виконує роль незмінного первинного ключа для реляційних зв'язків у межах системи, тоді як поле `email` використовується як унікальний логін для доступу. З міркувань кібербезпеки зберігання паролів у відкритому вигляді суворо виключено — замість цього у полі `passwordHash` утримується лише результат криптографічного хешування, що захищає облікові дані від компрометації навіть у разі витоку бази. Окремий архітектурний акцент зроблено на атрибуті `nickname`: оскільки саме це ім'я фігурує в публічних таблицях лідерів, на рівні схеми бази даних застосовано індекс з обмеженням унікальності, що гарантує відсутність дублікатів імен пілотів та забезпечує миттєвий пошук профілю (див. Рисунок 2.1).

					КП.ІПЗ-4121.ІЗ	Арк.
						15
Зм.	Арк.	№ докум.	Підпис	Дата		

users	
<u>_id</u>	ObjectId
email	varchar NN
passwordHash	varchar NN
nickname	varchar
age	integer
country	varchar

Рисунок 2.1 – Зображення структури профіля користувача

Центральним елементом архітектури виступає сутність сесії, яка слугує базовою одиницею для розрахунку рейтингів. Цей документ консолідує повний спектр інформації про заїзд: від метаданих про умови на трасі до деталізованого масиву кіл, що містить як «сірі», так і нормалізовані показники телеметрії.

Для коректної побудови таблиць лідерів застосовується принцип композитного ключа, що базується на поєднанні параметрів track (траса) та vehicle (автомобіль). Саме ця пара (наприклад, «Monza + Ferrari 296 GT3») визначає контекст для групування результатів. Крім того, задля масштабування системи впроваджено атрибут simulator. Він дозволяє інтегрувати різні платформи (такі як ACC або iRacing) у спільну базу даних, забезпечуючи логічне розмежування їхніх потоків даних (див Рисунок 2.2).

sessions	
<u>_id</u>	ObjectId
user	ObjectId
track	varchar NN
vehicle	varchar NN
simulator	varchar
date	datetime NN

Рисунок 2.2 – Зображення структури сесії

					КП.ІІЗ-4121.ІЗ	Арк.
						16
Зм.	Арк.	№ докум.	Підпис	Дата		

Ключовою метрикою продуктивності виступає поле `fastestLapTime` (див Рисунок 2.3), яке фіксує найкращий результат у секундах з плаваючою крапкою. Використання числового формату є стратегічним рішенням: воно дозволяє двигуну бази даних виконувати індексацію та сортування рейтингів на низькому рівні без додаткових витрат ресурсів на перетворення типів.

В основі підсистеми зберігання телеметрії лежить гібридна модель даних. Для масиву `laps` застосовано патерн вкладених документів (Embedded Documents) (див Рисунок 2.3).

sessions 	
<code>_id</code> 	ObjectId
<code>user</code> 	ObjectId
<code>track</code> 	varchar NN
<code>vehicle</code>	varchar NN
<code>simulator</code> 	varchar
<code>date</code>	datetime NN
<code>totalLaps</code>	integer
<code>fastestLapTime</code>	float
<code>fastestLapNum</code>	integer
<code>laps</code> 	json
<code>filenameLd</code>	varchar
<code>filenameLdx</code>	varchar

Рисунок 2.3 – Зображення метрик «`fastestLapTime`» та «`laps`»

Інтеграція повної хронології сесії безпосередньо в документ заїзду забезпечує атомарність операцій читання та дозволяє отримувати вичерпний перелік часів усіх кіл за один запит до бази даних, виключаючи накладні витрати на об'єднання (JOINS). Водночас, для ідентифікації пілота використано нормалізований підхід через посилання (Document Reference) у полі `user`. Такий архітектурний поділ дозволяє ефективно структурувати дані, логічно відокремлюючи динамічні результати заїздів від облікового запису користувача.

2.3 Проєктування алгоритмів серверної логіки

Архітектура програмного комплексу базується на розробці двох фундаментальних алгоритмічних моделей, що визначають логіку обробки даних у системі: конвеєра (pipeline) прийому телеметрії та механізму агрегації рейтингів. Проєктне рішення для підсистеми завантаження даних передбачає багатоступеневу модель обробки, де пріоритетом виступає безпека та цілісність даних. На етапі проєктування закладено механізм обов'язкової верифікації джерела запиту, що базується на криптографічній перевірці токенів доступу. Це архітектурне рішення дозволяє встановити жорстку асоціацію між завантаженим файлом та унікальним ідентифікатором користувача ще до початку обробки самого файлу, що концептуально унеможливлює підміну авторства результатів. Для роботи з бінарними даними спроєктовано стратегію потокової буферизації, яка передбачає тимчасове збереження вхідного потоку на фізичний носій замість утримання в оперативній пам'яті, що є критичним рішенням для забезпечення відмовостійкості системи при високих навантаженнях.

Для аналізу телеметрії обрано архітектурний патерн ізоляції обчислювальних процесів. Проєктом передбачено винесення ресурсомістких операцій декодування пропрієтарних форматів (MoTeC) у відокремлений модуль, реалізований на мові, що має нативну підтримку необхідних математичних бібліотек. Такий підхід дозволяє розпаралелити процеси введення-виведення основного сервера та обчислювальні операції аналізатора. Логічна модель валідації даних базується на пошуку маркерів хронометражу та розрахунку сегментів траси; при цьому спроєктовано механізм фільтрації "на вході", який відхиляє файли без коректно завершених кіл, запобігаючи забрудненню бази даних нерелевантною інформацією. Успішне проходження цього етапу ініціює збереження структурованого документа в базу даних та архівацію вихідного файлу, що забезпечує довгострокову доступність історії заїздів.

					КП.ІІЗ-4121.ІЗ	Арк.
						18
Зм.	Арк.	№ докум.	Підпис	Дата		

Проектування підсистеми рейтингів базується на відмові від статичного зберігання таблиць лідерів на користь моделі динамічної агрегації даних "на вимогу". Таке рішення дозволяє уникнути проблем із синхронізацією даних, гарантуючи користувачу актуальність відображеної інформації в реальному часі. Логіка побудови рейтингу спирається на використання конвеєрної обробки (Aggregation Pipeline), де послідовно застосовуються етапи фільтрації контексту (вибірка за трасою та авто) та сортування за ключовою метрикою часу. Ключовим елементом логічної схеми є алгоритм дедублікації результатів, який проектується для групування записів за унікальним ідентифікатором пілота з подальшою селекцією єдиного найкращого результату. Це забезпечує дотримання принципу спортивної справедливості, де в фінальній вибірці кожен учасник представлений лише один раз. Фінальний етап проектування передбачає використання реляційних зв'язків між колекціями для збагачення рейтингової таблиці публічними даними профілів користувачів, формуючи кінцеву структуру відповіді для клієнтського інтерфейсу.

2.4 Проектування архітектури REST API

Для забезпечення стандартизованої та масштабованої взаємодії між клієнтським додатком та серверною частиною розроблено специфікацію програмних інтерфейсів, що базується на архітектурному стилі REST та використовує протокол HTTP/1.1. Проектне рішення передбачає логічну сегрегацію кінцевих точок (endpoints) за функціональними доменами, виділяючи модуль авторизації для керування доступом та модуль управління сесіями для реалізації основної бізнес-логіки. Архітектура модуля авторизації спроектована для забезпечення безпечної ідентифікації користувачів та захисту ресурсів. Інтерфейс реєстрації розроблено з урахуванням необхідності валідації унікальності облікових записів на рівні бази даних, що запобігає дублюванню нікнеймів та електронних адрес. Для процесу аутентифікації обрано стратегію безстанової (stateless) взаємодії, де підтвердженням успішного входу є генерація JWT-токена.

					КП.ІПЗ-4121.ІЗ	Арк.
						18
Зм.	Арк.	№ докум.	Підпис	Дата		

Таке рішення дозволяє розвантажити сервер від зберігання сесій у пам'яті та забезпечує масштабованість системи, оскільки токен виступає універсальною перепусткою до захищених маршрутів.

Проектування модуля телеметрії спрямоване на надійну передачу файлів та зручний доступ до статистики. Для завантаження результатів використовується захищений маршрут на базі стандарту multipart/form-data, який дозволяє серверу приймати бінарні файли разом із необхідними метаданими. Обов'язковою умовою є наявність авторизації, що гарантує автоматичну прив'язку результату до профілю пілота без додаткових ручних дій. Для отримання рейтингів розроблено гнучкий API з підтримкою фільтрації через параметри запиту. Це дозволяє клієнту миттєво отримувати список лідерів для конкретної траси та автомобіля, не завантажуючи зайву інформацію з бази даних. Окрему увагу приділено функції доступу до первинних даних. Система передбачає захищений механізм, що дозволяє завантажити оригінальний файл телеметрії за його ID. Це рішення є критичним для забезпечення чесності змагань: у спірних ситуаціях судді можуть отримати вихідний файл та детально проаналізувати його у спеціалізованому програмному забезпеченні на предмет шахрайства. Таким чином, API виконує роль не лише транспорту даних, а й інструменту контролю за дотриманням правил.

					КП.ІІЗ-4121.ІЗ	Арк.
						20
Зм.	Арк.	№ докум.	Підпис	Дата		

3 ПРОГРАМНА РЕАЛІЗАЦІЯ СИСТЕМИ

3.1 Реалізація підсистеми доступу до даних

Для роботи з базою даних у системі використовується бібліотека **Mongoose ODM**. Її обрано для зручної взаємодії **TypeScript** із документо-орієнтованою базою даних **MongoDB**. **Mongoose** виступає проміжним шаром між застосунком і базою даних, автоматично виконуючи валідацію даних, приведення типів і формування запитів, що забезпечує більш надійну та зрозумілу роботу з даними порівняно з нативним драйвером.

Налаштування з'єднання з кластером **MongoDB** виконується в кореневому модулі **AppModule** за допомогою асинхронного методу **MongooseModule.forRootAsync**. Такий підхід дозволяє динамічно отримувати рядок підключення з змінних середовища через **ConfigService**, не зберігаючи чутливі дані безпосередньо в коді. Це відповідає принципам безпеки методології **12-Factor App**. Для стабільної роботи під високим навантаженням використовується пул з'єднань, який забезпечує ефективну обробку паралельних запитів користувачів (див. Рисунок 3.1).

```
@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    MongooseModule.forRootAsync({
      imports: [ConfigModule],
      inject: [ConfigService],
      useFactory: (configService: ConfigService) => ({
        uri: configService.get<string>('MONGO_URI'),
      }),
    }),
    AuthModule,
    UsersModule,
    SessionsModule,
  ],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule { }
```

Рисунок 3.1 - Реалізація асинхронного підключення до бази даних у модулі додатка

					КП.ІПЗ-4121.ПЗ	Арк.
						20
Зм.	Арк.	№ докум.	Підпис	Дата		

Моделювання даних у системі реалізовано за допомогою класів-схем із використанням декораторів `@Schema` та `@Prop`. У системі визначено дві основні моделі — `User` та `Session`. Для користувачів налаштовано унікальні обмеження на електронну пошту та нікнейм, що запобігає створенню дубльованих облікових записів.

Інформація про кола заїзду зберігається у вигляді вкладеного масиву, що дає змогу тримати всю історію сесії в одному документі та підвищує швидкість читання даних (див. Рисунок 3.2). Усі схеми зареєстровані в контейнері залежностей NestJS через метод `forFeature`, що забезпечує їх використання в сервісах бізнес-логіки.

```
@Prop({ required: true })
track: string;

@Prop({ required: true })
vehicle: string;

@Prop({ required: true, default: 'Assetto Corsa Competizione' })
simulator: string;

@Prop({ required: true })
date: Date;

@Prop()
totalLaps: number;

@Prop()
fastestLapTime: number;

@Prop()
fastestLapNum: number;

@Prop({ type: [Object] })
laps: any[];

@Prop()
filenameId: string;

@Prop()
```

Рисунок 3.2 - Декларативний опис схеми даних сесії засобами бібліотеки
Mongoose

Модель сесій побудована з урахуванням зв'язку між даними: поле автора зберігає посилання (`ObjectId`) на користувача. Це дозволяє отримувати повну інформацію про пілота в межах одного запиту за допомогою методу `populate()`, аналогічно до операції JOIN у реляційних базах даних.

					КП.ІІЗ-4121.ІЗ	Арк.
						21
Зм.	Арк.	№ докум.	Підпис	Дата		

3.2 Програмна реалізація обробки телеметрії

Основу бізнес-логіки системи становить модуль аналізу даних, побудований за гібридним підходом. Загальну координацію роботи виконує середовище Node.js, а складні обчислення передано Python-скриптам. Взаємодія між ними здійснюється через стандартний механізм запуску дочірніх процесів.

Під час завантаження файлу контролер асинхронно запускає окремий процес операційної системи та передає йому шлях до файлу. Це дозволяє виконувати обчислення окремо від основного потоку Node.js і запобігає блокуванню роботи сервера, що забезпечує стабільну роботу API навіть за великої кількості запитів. Результати обробки передаються назад у вигляді JSON через стандартний потік виводу та зберігаються в базі даних (див. Рисунок 3.3).

```
const scriptPath = path.resolve(process.cwd(), 'src/convert_motec.py');
try {
  const { stdout, stderr } = await execAsync(`python3 "${scriptPath}" "${ldPath}" "metadata_only"`);
  console.log("Python output:", stdout);

  let pythonMeta = {};
  try {
    pythonMeta = JSON.parse(stdout);
  } catch (e) {
    console.error("Failed to parse python JSON output", stdout);
  }

  return {
    message: 'Data extracted successfully',
    summary: ldxInfo || {
      totallaps: pythonMeta['laps_info'] ? pythonMeta['laps_info'].length : 0,
      fastestlap: pythonMeta['fastest_lap'] ? pythonMeta['fastest_lap']['lap_num'] : null,
      fastestTime: pythonMeta['fastest_lap'] ? pythonMeta['fastest_lap']['time'] : null,
      ...pythonMeta['metadata']
    },
    laps: pythonMeta['laps_info'] || [],
    metadata: pythonMeta['metadata'] || {},
    ldx_source: !!ldxInfo
  };
} catch (e) {
  return {
    message: 'Processing failed',
    error: e.message
  };
}
```

Рисунок 3.3 - Реалізація асинхронного запуску Python-скрипта в контролері

					КП.ІПЗ-4121.ПЗ	Арк.
						23
Зм.	Арк.	№ докум.	Підпис	Дата		

Важливим етапом є вирівнювання даних у часі, оскільки різні датчики працюють з різною частотою. Після цього дані розбиваються на окремі кола за спеціальними маркерами, перевіряється їх коректність і формується підсумковий звіт з інформацією про пілота, трасу та автомобіль(див. Рисунок 3.4).

```
# Base dataframe with all data
dfs = []
for chan in valid_channels:
    duration = len(chan.data) / chan.freq
    time = np.linspace(0, duration, len(chan.data), endpoint=False)
    time = np.round(time, 6)
    series = pd.Series(data=chan.data, index=time, name=chan.name)
    dfs.append(series)

df = pd.concat(dfs, axis=1)
df.sort_index(inplace=True)
df.index.name = 'Time'
df.fillna(method='ffill', inplace=True)
df.fillna(method='bfill', inplace=True)
```

Рисунок 3.4 - Використання Pandas для структурування даних телеметрії

Безпосередній аналіз виконується Python-скриптом `convert_motec.py`. Для зчитування телеметричних даних із бінарного формату `.ld` використовується спеціалізований парсер, після чого дані обробляються за допомогою бібліотеки Pandas. Телеметрія перетворюється у структурований вигляд, що дозволяє швидко виконувати обчислення над великими обсягами даних.

3.3 Реалізація REST API інтерфейсів

Взаємодія клієнтських застосунків із серверною частиною реалізована за допомогою REST API, побудованого на фреймворку NestJS. Архітектура передбачає чітке розділення між рівнем контролерів і рівнем бізнес-логіки, що відповідає принципу єдиної відповідальності та спрощує підтримку й розширення коду.

Основним компонентом для роботи з телеметричними даними є `SessionsController`. Маршрути API визначаються декларативно за допомогою декораторів `@Controller`, `@Get` та `@Post`, що дозволяє чітко структурувати доступні ендпоінти та зробити код більш наочним.

					КП.ІПЗ-4121.ПЗ	Арк.
						24
Зм.	Арк.	№ докум.	Підпис	Дата		

Для захисту критичних маршрутів використовується механізм Guard із JWT-аутентифікацією. Захищені ендпоінти, зокрема завантаження файлів і доступ до телеметрії, перевіряють наявність та дійсність токена доступу. У разі некоректного токена запит автоматично відхиляється з помилкою 401 Unauthorized, не передаючись на подальшу обробку, що зменшує навантаження на сервер (див. Рисунок 3.5).

```
@UseGuards(JwtAuthGuard)
@Get('best')
async getBestSessions(
    @Request() req,
    @Query('track') track: string,
    @Query('vehicle') vehicle?: string,
    @Query('simulator') simulator?: string
) {
    if (!track) {
        return { message: 'Track parameter is required' };
    }
    return this.sessionsService.findBestSessions(track, vehicle, simulator);
}
```

Рисунок 3.5 - Реалізація REST-контролера для отримання рейтингових даних

Завантаження файлів реалізовано з використанням механізму прийому даних у форматі multipart/form-data. API підтримує одночасне завантаження основного файлу телеметрії та додаткового файлу розмітки. Перед передачею даних у сервісний шар виконується базова перевірка наявності файлів.

Для обробки вхідних параметрів застосовується валідація на рівні контролерів. Фільтрація даних здійснюється через параметри запиту, а числові значення автоматично приводяться до потрібних типів. Це знижує ризик помилок під час виконання запитів до бази даних і підвищує стабільність роботи API (див. Рисунок 3.6).

					КП.ІІЗ-4121.ІЗ	Арк.
						25
Зм.	Арк.	№ докум.	Підпис	Дата		

```
@UseGuards(JwtAuthGuard)
@Post('upload')
@UseInterceptors(FileFieldsInterceptor([
  { name: 'file', maxCount: 1 },
  { name: 'ldx', maxCount: 1 },
]))
async uploadFile(@UploadedFiles() files: { file?: Express.Multer.File[], ldx?: Express.Multer.File[] }, @Body() body, @Request() req) {
```

Рисунок 3.6 - Реалізація захищеного методу завантаження файлів із використанням інтерцепторів

Додатково передбачено логування ключових етапів обробки файлів для полегшення відстеження помилок та аналізу продуктивності системи. Такий підхід забезпечує високу надійність та готовність API до роботи з великими обсягами телеметричних даних.

3.4 Тестування та валідація працездатності системи

Для підтвердження коректної роботи розробленого програмного комплексу було проведено комплекс інтеграційних тестів, спрямованих на перевірку повного циклу взаємодії користувача з системою — від автентифікації до отримання аналітичних результатів. Тестування виконувалося з використанням інструменту Postman, який дозволяє моделювати HTTP-запити клієнтських застосунків та аналізувати відповіді сервера в різних сценаріях використання.

На початковому етапі було перевірено роботу підсистеми безпеки та механізмів контролю доступу. Під час звернення до захищених API-ендпоінтів без токена доступу система коректно повертала код помилки 401 Unauthorized, що підтверджує ефективність реалізованих Guard-механізмів. Після успішної автентифікації через ендпоінт /auth/login сервер генерував JWT-токен, який надалі використовувався для авторизації запитів до захищених ресурсів (див. Рисунок 3.7).

					КП.ІІЗ-4121.ІЗ	Арк.
						26
Зм.	Арк.	№ докум.	Підпис	Дата		

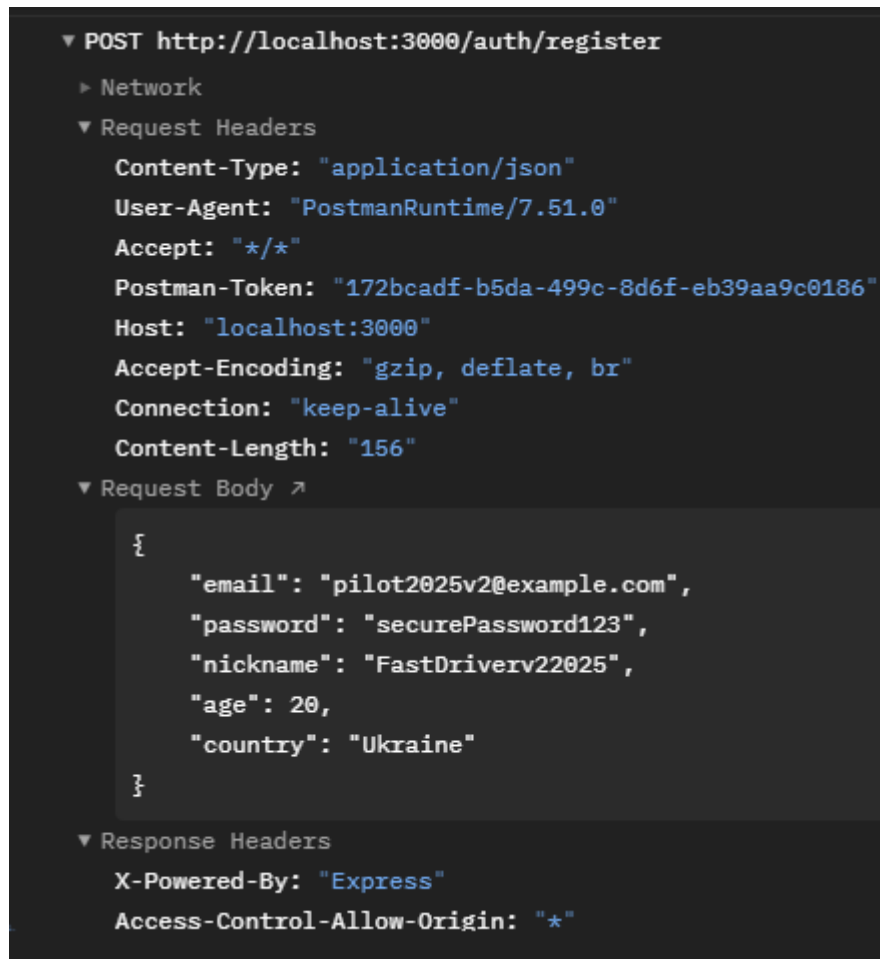


Рисунок 3.7 - Успішна аутентифікація користувача та отримання токена доступу

Основну увагу було приділено перевірці процесу завантаження та обробки телеметричних даних. У межах тестування було виконано завантаження еталонного файлу формату .ld, отриманого з гоночного симулятора Assetto Corsa Competizione(див. Рисунок 3.8). Система коректно обробила запит у форматі multipart/form-data, виконала аналіз вхідних даних і автоматично ідентифікувала ключові метадані сесії, зокрема інформацію про автомобіль, трасу та пілота. У відповідь сервер повернув структуровані результати у форматі JSON, придатні для подальшого використання клієнтськими застосунками.

					КП.ІПЗ-4121.ПЗ	Арк.
						27
Зм.	Арк.	№ докум.	Підпис	Дата		

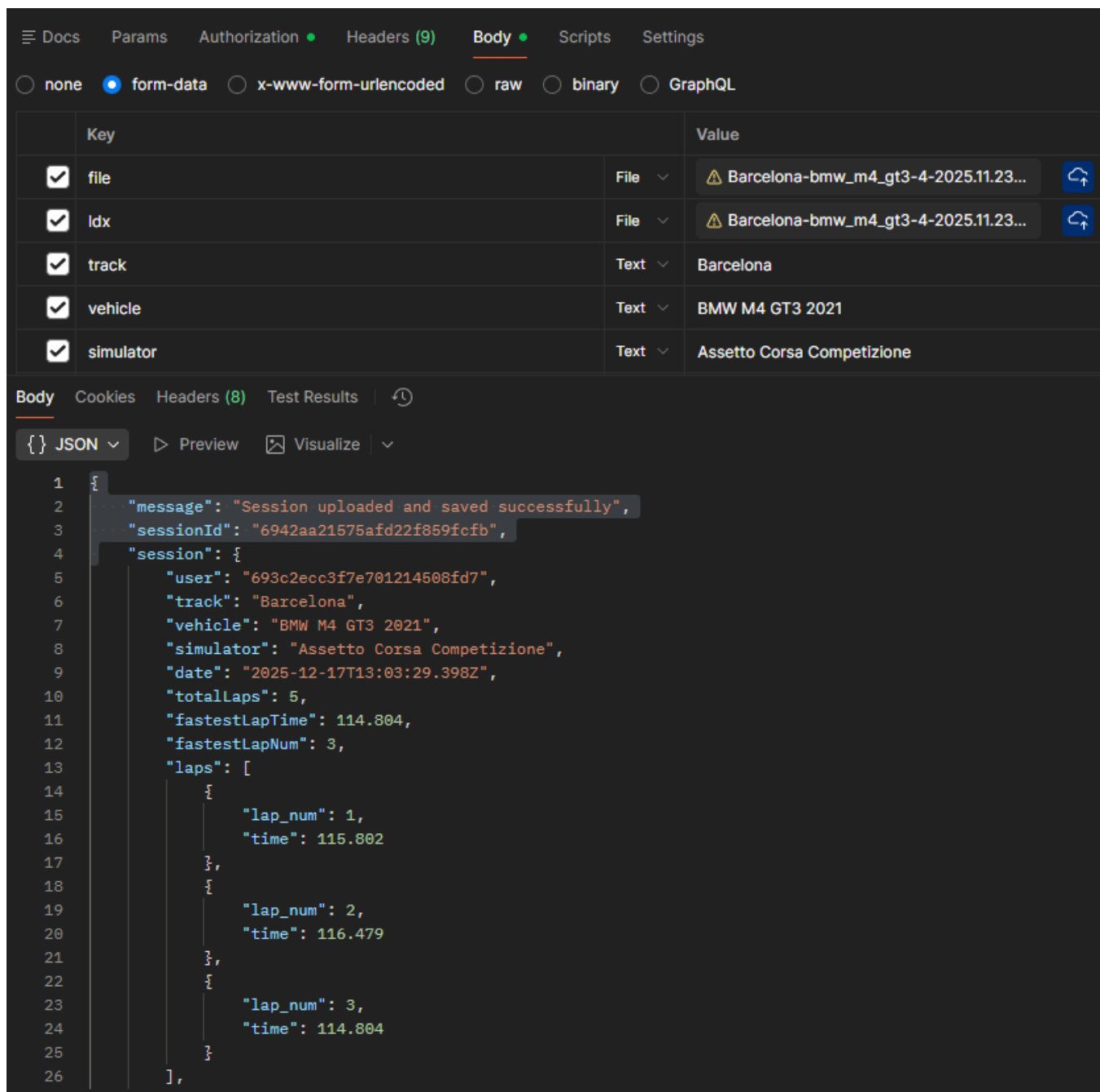


Рисунок 3.8 - Результат успішної обробки телеметричного файлу (.ld) та сформована JSON-відповідь сервера

Це підтверджує стабільність та надійність реалізованого API у роботі з різними файлами телеметрії. Крім того, отримані дані можна безпосередньо інтегрувати у системи візуалізації та рейтингові таблиці, що підвищує практичну цінність розробленого рішення.

					КП.ІПЗ-4121.ПЗ	Арк.
						27
Зм.	Арк.	№ докум.	Підпис	Дата		

ВИСНОВКИ

У ході виконання курсової роботи було спроектовано та реалізовано серверну частину вебзастосунку для аналізу телеметричних даних автосимуляторів. Для розробки back-end частини було обрано фреймворк NestJS, який забезпечив побудову модульної, масштабованої та легко розширюваної архітектури API.

Ключовим результатом роботи стала реалізація механізму автоматизованої обробки бінарних файлів телеметрії формату MoTeC (.ld). У процесі розробки було виявлено обмеження початкового підходу, заснованого на використанні зовнішніх Python-скриптів, зокрема складність налаштування середовища виконання та залежність від сторонніх бібліотек. З метою усунення цих недоліків було виконано рефакторинг системи з переходом на використання нативної бібліотеки Node.js `@rflafla/motec-ld-reader`.

- Застосування даного рішення дозволило досягти таких переваг:
- підвищення продуктивності за рахунок обробки телеметричних даних безпосередньо в оперативній пам'яті сервера без запуску дочірніх процесів;
- спрощення процесу розгортання системи завдяки відсутності потреби у встановленні Python та додаткових залежностей;
- забезпечення кросплатформності та коректної роботи як у середовищі Windows, так і в Linux.

Окрім цього, було реалізовано набір REST-контролерів для завантаження файлів, валідації вхідних даних та збереження метаданих заїздів, зокрема інформації про пілота, автомобіль, трасу та часи кіл, у базу даних. Розроблений API є повністю готовим до інтеграції з клієнтською частиною застосунку та формує надійну основу для подальшого розвитку системи, включаючи розширення аналічного функціонала, наприклад, додавання деталізованих графіків телеметрії.

					КП.ІІЗ-4121.ІЗ	Арк.
						29
Зм.	Арк.	№ докум.	Підпис	Дата		

СПИСОК ВИКОРИСТАНИХ ДЕРЕЛ

1. NestJS Documentation – Official documentation. URL: (Дата звернення: 12.10.2025).
2. Node.js Documentation – Official Node.js documentation. URL: <https://nodejs.org/en/docs> (Дата звернення: 12.10.2025).
3. MongoDB Documentation – Official MongoDB documentation. URL: <https://www.mongodb.com/docs> (Дата звернення: 15.10.2025).
4. Mongoose ODM Documentation – Official Mongoose documentation. URL: <https://mongoosejs.com/docs> (Дата звернення: 15.10.2025).
5. JWT.io – JSON Web Tokens Introduction. URL: <https://jwt.io/introduction> (Дата звернення: 16.10.2025).
6. Postman Documentation – API testing and development documentation. URL: <https://learning.postman.com/docs> (Дата звернення: 30.10.2025).
7. MoTeC – Official MoTeC documentation and telemetry overview. URL: <https://www.motec.com.au> Дата звернення: (10.11.2025).
8. Assetto Corsa Competizione – Official simulator documentation. URL: <https://www.assettocorsa.it/competizione> (Дата звернення: 10.11.2025).
9. @rflafla/motec-ld-reader – Node.js library for reading MoTeC .ld files. URL: <https://www.npmjs.com/package/@rflafla/motec-ld-reader> (Дата звернення: 11.11.2025).

					КП.ІПЗ-4121.ПЗ	Арк.
						30
Зм.	Арк.	№ докум.	Підпис	Дата		

Додаток А

А.1 Реалізація підсистеми доступу до даних

А.1.1 Реалізація асинхронного підключення до бази даних (файл `app.module.ts`):

```
import { Module } from '@nestjs/common';
import { ConfigModule, ConfigService } from '@nestjs/config';
import { MongooseModule } from '@nestjs/mongoose';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { AuthModule } from './auth/auth.module';
import { UsersModule } from './users/users.module';
import { SessionsModule } from './sessions/sessions.module';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    MongooseModule.forRootAsync({
      imports: [ConfigModule],
      inject: [ConfigService],
      useFactory: (configService: ConfigService) => ({
        uri: configService.get<string>('MONGO_URI'),
      }),
    }),
    AuthModule,
    UsersModule,
    SessionsModule,
  ],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule { }
```

A.1.2 Декларативний опис схеми даних сесії (файл session.schema.ts):

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { Document, Schema as MongooseSchema } from 'mongoose';
import { User } from './user.schema';

export type SessionDocument = Session & Document;

@Schema()
export class Session {
  @Prop({ type: MongooseSchema.Types.ObjectId, ref: 'User', required: true })
  user: User;

  @Prop({ required: true })
  track: string;

  @Prop({ required: true })
  vehicle: string;

  @Prop({ required: true, default: 'Assetto Corsa Competizione' })
  simulator: string;

  @Prop({ required: true })
  date: Date;

  @Prop()
  totalLaps: number;

  @Prop()
  fastestLapTime: number;

  @Prop()
  fastestLapNum: number;

  @Prop({ type: [Object] })
  laps: any[];

  @Prop()
  filenameLd: string;

  @Prop()
  filenameLdx: string;
}

export const SessionSchema = SchemaFactory.createForClass(Session);
```


A.1.3 Модель User та валідація(файл user.schema.ts):

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { Document } from 'mongoose';

export type UserDocument = User & Document;

@Schema()
export class User {
  @Prop({ required: true, unique: true })
  email: string;

  @Prop({ required: true })
  passwordHash: string;

  @Prop({ unique: true })
  nickname: string;

  @Prop()
  age: number;

  @Prop()
  country: string;
}

export const UserSchema = SchemaFactory.createForClass(User);
```

A.1.4 Використання populate() для зв'язків(файл sessions.service.ts)

```
.populate('user', 'email nickname age country')
```

A.2 Програмна реалізація обробки телеметрії

A.2.1 Запуск Python-скрипта через `child_process`(файл `sessions.controller.ts`):

```
import { Controller, Post, UseInterceptors, UploadedFiles, Body, UseGuards, Request,
Get, Query, Res, Param, NotFoundException, BadRequestException } from
'@nestjs/common';
import { FileFieldsInterceptor } from '@nestjs/platform-express';
import { SessionsService } from '../sessions.service';
import { JwtAuthGuard } from '../auth/jwt-auth.guard';
import * as fs from 'fs';
import * as path from 'path';
import { exec } from 'child_process';
import { promisify } from 'util';

const execAsync = promisify(exec);

@Controller('sessions')
export class SessionsController {
  constructor(private sessionsService: SessionsService) {}

  @UseGuards(JwtAuthGuard)
  @Post('upload')
  @UseInterceptors(FileFieldsInterceptor([
    { name: 'file', maxCount: 1 },
    { name: 'idx', maxCount: 1 },
  ]))
  async uploadFile(@UploadedFiles() files: { file?: Express.Multer.File[], idx?:
Express.Multer.File[] }, @Body() body, @Request() req) {

    // Validate Track and Vehicle if provided in body
    const tracksPath = path.resolve(process.cwd(), 'src/data/tracks.json');
    const carsPath = path.resolve(process.cwd(), 'src/data/cars.json');
    const autosimsPath = path.resolve(process.cwd(), 'src/data/autosims.json');

    let validTracks: string[] = [];
    let validCars: string[] = [];
    let validSimulators: string[] = [];

    if (fs.existsSync(tracksPath)) {
      const tracksData = JSON.parse(fs.readFileSync(tracksPath, 'utf8'));
      validTracks = tracksData.map((t: any) => t.name);
    }

    if (fs.existsSync(carsPath)) {
      const carsData = JSON.parse(fs.readFileSync(carsPath, 'utf8'));
      // Flatten the categories to get a simple list of cars
      validCars = Object.values(carsData).flat().map((c: any) => c.name);
    }
  }
}
```

```

if (fs.existsSync(autosimsPath)) {
    const autosimsData = JSON.parse(fs.readFileSync(autosimsPath, 'utf8'));
    validSimulators = autosimsData.map((s: any) => s.name);
}

if (body.track && !validTracks.includes(body.track)) {
    throw new BadRequestException(`Invalid track: ${body.track}.`);
}

if (body.vehicle && !validCars.includes(body.vehicle)) {
    throw new BadRequestException(`Invalid vehicle: ${body.vehicle}`);
}

if (body.simulator && !validSimulators.includes(body.simulator)) {
    throw new BadRequestException(`Invalid simulator: ${body.simulator}`);
}

const filesDir = path.resolve(process.cwd(), 'files');
if (!fs.existsSync(filesDir)) {
    fs.mkdirSync(filesDir, { recursive: true });
}

const file = files.file ? files.file[0] : null;
const idxFile = files.idx ? files.idx[0] : null;

if (!file) {
    return { message: 'No .ld file uploaded' };
}

// Use user ID to make unique filenames or folders if needed, for now overwrite
or simple unique
const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
const ldPath = path.resolve(filesDir, `source-${uniqueSuffix}.ld`);
fs.writeFileSync(ldPath, file.buffer);

let idxInfo: any = null;
if (idxFile) {
    const idxPath = path.resolve(filesDir, `source-${uniqueSuffix}.idx`);
    fs.writeFileSync(idxPath, idxFile.buffer);

    const idxContent = idxFile.buffer.toString();
    const extractValue = (key: string) => {
        const match = idxContent.match(new RegExp(`<String Id="${key}"
Value="([^\"]+)"`));
        return match ? match[1] : null;
    };

    const totalLaps = extractValue('Total Laps');
    const fastestTime = extractValue('Fastest Time');
    const fastestLap = extractValue('Fastest Lap');

    const beaconMatches =

```

```

IdxContent.matchAll(/<Marker.*?ClassName="BCN".*?Time="([^\"]+)"/g);
const beaconTimes: number[] = [];
for (const match of beaconMatches) {
  const timeMicro = parseFloat(match[1]);
  if (!isNaN(timeMicro)) {
    beaconTimes.push(timeMicro / 1000000);
  }
}

const IdxLaps: any[] = [];
for (let i = 0; i < beaconTimes.length - 1; i++) {
  const lapTime = beaconTimes[i + 1] - beaconTimes[i];
  IdxLaps.push({
    lap_num: i + 1,
    time: parseFloat(lapTime.toFixed(3))
  });
}

if (fastestTime && fastestLap) {
  IdxInfo = {
    source: 'Idx',
    totalLaps: totalLaps ? parseInt(totalLaps) : 0,
    fastestLap: fastestLap ? parseInt(fastestLap) : null,
    fastestTime: fastestTime,
    driver: 'Unknown (from LDX)',
    vehicle: 'Unknown (from LDX)',
    laps: IdxLaps
  };
}

}

try {
  const { LdData } = require('@rflafla/motec-ld-reader');
  const IdData = LdData.fromBuffer(file.buffer);

  // Extract metadata
  const metadata = {
    driver: IdData.head.driver,
    vehicle: IdData.head.vehicleid,
    venue: IdData.head.venue,
    date: IdData.head.datetime,
    comment: IdData.head.short_comment,
    event: IdData.head.event ? IdData.head.event.name : "",
    session: IdData.head.event ? IdData.head.event.session : ""
  };

  const summary = IdxInfo || {
    totalLaps: 0,
    fastestLap: null,
    fastestTime: null,
  }
}

```

```

...metadata
};

const laps = idxInfo ? idxInfo.laps : [];

const track = body.track || summary.venue || 'Unknown Track';
const vehicle = body.vehicle || summary.vehicle || 'Unknown Vehicle';

const date = summary.date !== 'Unknown' ? new Date(summary.date ||
Date.now()) : new Date();

const parseTimeString = (timeStr: string | number): number => {
  if (typeof timeStr === 'number') return timeStr;
  if (!timeStr) return 0;

  const floatVal = parseFloat(timeStr);
  if (!isNaN(floatVal) && timeStr.indexOf('.') === -1) return floatVal;

  const parts = timeStr.split(':');
  if (parts.length === 2) {
    const minutes = parseFloat(parts[0]);
    const seconds = parseFloat(parts[1]);
    return (minutes * 60) + seconds;
  }
  return 0;
};

const parsedFastestTime = parseTimeString(summary.fastestTime);

// Create Session Record
const sessionPayload = {
  user: req.user.userId,
  track: track,
  vehicle: vehicle,
  simulator: body.simulator || 'Assetto Corsa Competizione',
  date: date,
  totalLaps: summary.totalLaps,
  fastestLapTime: parsedFastestTime > 0 ? parsedFastestTime : 0,
  fastestLapNum: summary.fastestLap,
  laps: laps,
  filenameLd: `source-${uniqueSuffix}.ld`,
  filenameLdx: idxFile ? `source-${uniqueSuffix}.ldx` : null
};

const savedSession = await this.sessionsService.create(sessionPayload);

return {
  message: 'Session uploaded and saved successfully',
  sessionId: savedSession._id,
  session: savedSession
}

```

```

};

    } catch (e) {
        // cleanup if fail?
        return {
            message: 'Processing failed',
            error: e.message
        };
    }
}

@UseGuards(JwtAuthGuard)
@Get('recent')
async getRecentSessions(
    @Query('limit') limitArg: string,
    @Query('track') track?: string,
    @Query('vehicle') vehicle?: string,
    @Query('simulator') simulator?: string,
    @Query('userId') userId?: string
) {
    const limit = parseInt(limitArg) || 20;
    const filters: any = {};
    if (track) filters.track = track;
    if (vehicle) filters.vehicle = vehicle;
    if (simulator) filters.simulator = simulator;
    if (userId) filters.user = userId;

    return this.sessionsService.findRecentSessions(limit, filters);
}

@UseGuards(JwtAuthGuard)
@Get('best')
async getBestSessions(
    @Request() req,
    @Query('track') track: string,
    @Query('vehicle') vehicle?: string,
    @Query('simulator') simulator?: string
) {
    if (!track) {
        return { message: 'Track parameter is required' };
    }
    return this.sessionsService.findBestSessions(track, vehicle, simulator);
}

@Get('cars')
getCars() {
    const carsPath = path.resolve(process.cwd(), 'src/data/cars.json');
    if (fs.existsSync(carsPath)) {
        const data = fs.readFileSync(carsPath, 'utf8');
        return JSON.parse(data);
    }
}

```

```

return { message: 'Cars data not found' };
}

@Get('tracks')
getTracks() {
  const tracksPath = path.resolve(process.cwd(), 'src/data/tracks.json');
  if (fs.existsSync(tracksPath)) {
    const data = fs.readFileSync(tracksPath, 'utf8');
    return JSON.parse(data);
  }
  return { message: 'Tracks data not found' };
}

@Get('autosims')
getAutosims() {
  const autosimsPath = path.resolve(process.cwd(), 'src/data/autosims.json');
  if (fs.existsSync(autosimsPath)) {
    const data = fs.readFileSync(autosimsPath, 'utf8');
    return JSON.parse(data);
  }
  return { message: 'Autosims data not found' };
}

@UseGuards(JwtAuthGuard)
@Get('download/:id/:type')
async downloadFile(@Request() req, @Param('id') id: string, @Param('type') type:
string, @Res() res) {
  const session = await this.sessionsService.findOne(id);
  if (!session) {
    throw new NotFoundException('Session not found');
  }

  // Check local files dir
  const filesDir = path.resolve(process.cwd(), 'files');
  let filename: string | null = null;

  if (type === 'ld') filename = session.filenameLd;
  else if (type === 'ldx') filename = session.filenameLdx;

  if (!filename) {
    throw new NotFoundException('File not found for this session');
  }

  const filePath = path.resolve(filesDir, filename);
  if (!fs.existsSync(filePath)) {
    throw new NotFoundException('File on disk not found');
  }

  return res.download(filePath);
}
}

```

A.2.2 Скрипт аналізу даних(файл convert_motec.py):

```

import sys
import os
import pandas as pd
import numpy as np

# Add lib to path so we can import ldpaser
sys.path.append(os.path.join(os.path.dirname(__file__), '../lib/ldparser'))
try:
    from ldpaser import ldData
except ImportError:
    # helper for when running from different cwd
    sys.path.append(os.path.join(os.getcwd(), 'lib/ldparser'))
    from ldpaser import ldData

def convert(input_path, output_path):
    sys.stderr.write(f"Converting {input_path} to {output_path}...\n")
    if not os.path.exists(input_path):
        sys.stderr.write(f"Error: File {input_path} not found\n")
        sys.exit(1)

    try:
        data = ldData.fromfile(input_path)
    except Exception as e:
        sys.stderr.write(f"Error parsing file: {e}\n")
        sys.exit(1)

    # Filter out empty channels and ensure freq > 0
    valid_channels = []
    for chan in data.channs:
        if len(chan.data) > 0 and chan.freq > 0:
            valid_channels.append(chan)

    if not valid_channels:
        sys.stderr.write("No valid data found in Motec file\n")
        sys.exit(1)

    # Base dataframe with all data
    dfs = []
    for chan in valid_channels:
        duration = len(chan.data) / chan.freq
        time = np.linspace(0, duration, len(chan.data), endpoint=False)
        time = np.round(time, 6)
        series = pd.Series(data=chan.data, index=time, name=chan.name)
        dfs.append(series)

    df = pd.concat(dfs, axis=1)
    df.sort_index(inplace=True)
    df.index.name = 'Time'
    df.fillna(method='ffill', inplace=True)

```



```
df.fillna(method='bfill', inplace=True)
```

```
# Metadata collection
```

```
metadata = {
```

```
    'driver': data.head.driver if data.head else 'Unknown',
```

```
    'vehicle': data.head.vehicleid if data.head else 'Unknown',
```

```
    'venue': data.head.venue if data.head else 'Unknown',
```

```
    'date': str(data.head.datetime) if data.head else 'Unknown',
```

```
    'comment': data.head.short_comment if data.head else "",
```

```
    'event': data.head.event.name if data.head and data.head.event else "",
```

```
    'session': data.head.event.session if data.head and data.head.event else ""
```

```
}
```

```
laps_info = []
```

```
# --- Lap Analysis ---
```

```
# Try to find a 'Lap Number' or 'Beacon' channel to split laps
```

```
lap_col = None
```

```
possible_names = ['Lap Number', 'Lap_Number', 'Lap', 'Beacon']
```

```
for col in df.columns:
```

```
    if col in possible_names:
```

```
        lap_col = col
```

```
        break
```

```
real_laps = []
```

```
if lap_col:
```

```
    df['LapInt'] = df[lap_col].round().astype(int)
```

```
    for lap_num, lap_data in df.groupby('LapInt'):
```

```
        if lap_num < 0: continue
```

```
        start_time = lap_data.index.min()
```

```
        end_time = lap_data.index.max()
```

```
        duration = end_time - start_time
```

```
        if duration < 1.0: continue
```

```
        real_laps.append({
```

```
            'lap_num': int(lap_num),
```

```
            'time': float(duration),
```

```
            'data': lap_data
```

```
        })
```

```
        laps_info.append({
```

```
            'lap_num': int(lap_num),
```

```
            'time': float(duration)
```

```
        })
```

```
else:
```

```
    # Single lap fallback
```

```
    sys.stderr.write("Warning: No Lap Number channel found. Treating as single lap.\n")
```

```
    duration = df.index.max() - df.index.min()
```

```

laps_info.append({
    'lap_num': 1,
    'time': float(duration)
})
real_laps.append({
    'lap_num': 1,
    'time': float(duration),
    'data': df
})

# JSON Output mode
if output_path == 'metadata_only':
    import json
    result = {
        'metadata': metadata,
        'laps_info': laps_info,
        'fastest_lap': min(laps_info, key=lambda x: x['time']) if laps_info else None
    }
    print(json.dumps(result))
    sys.exit(0)

# Save CSVs if not metadata only
if real_laps:
    fastest_lap = min(real_laps, key=lambda x: x['time'])
    sys.stderr.write(f"Fastest Lap: {fastest_lap['lap_num']} time: {fastest_lap['time']:.3f}s\n")

    fastest_lap_path = output_path.replace('.csv', '_fastest.csv')
    save_csv(fastest_lap['data'], fastest_lap_path, data.head,
            extra_meta=[f"Lap Number: {fastest_lap['lap_num']}", f"Lap Time: {fastest_lap['time']:.3f}"])

    save_csv(df, output_path, data.head, extra_meta=["Type: Full Session"])

def save_csv(df, path, head, extra_meta=[]):
    try:
        with open(path, 'w', encoding='utf-8') as f:
            if head:
                f.write(f"# Driver: {head.driver}\n")
                f.write(f"# Vehicle: {head.vehicleid}\n")
                f.write(f"# Venue: {head.venue}\n")
                f.write(f"# Date: {head.datetime}\n")
                f.write(f"# Comment: {head.short_comment}\n")
            if head.event:
                f.write(f"# Event: {head.event.name}\n")
                f.write(f"# Session: {head.event.session}\n")

            for meta in extra_meta:
                f.write(f"# {meta}\n")

            f.write("# ----- \n")

```

```
df.to_csv(f)
    sys.stderr.write(f"Success: Created {path}\n")
except Exception as e:
    sys.stderr.write(f"Error writing CSV {path}: {e}\n")

if __name__ == "__main__":
    if len(sys.argv) < 3:
        sys.stderr.write("Usage: python3 convert_motec.py <input.ld> <output.csv>\n")
        sys.exit(1)

    convert(sys.argv[1], sys.argv[2])
```

А.3 Реалізація REST API інтерфейсів

А.3.1 Реалізація REST-контролера для отримання рейтингових даних(файл sessions.controller.ts):

```
@Controller('sessions')
export class SessionsController {
  constructor(private sessionsService: SessionsService) {}

  @UseGuards(JwtAuthGuard)
  @Get('best')
  async getBestSessions(
    @Request() req,
    @Query('track') track: string,
    @Query('vehicle') vehicle?: string,
    @Query('simulator') simulator?: string
  ) {
    if (!track) {
      return { message: 'Track parameter is required' };
    }
    return this.sessionsService.findBestSessions(track, vehicle, simulator);
  }
}
```

А.3.2 Завантаження файлів з інтерцепторами(sessions.controller.ts):

```
@UseGuards(JwtAuthGuard)
@Post('upload')
@UseInterceptors(FileFieldsInterceptor([
  { name: 'file', maxCount: 1 },
  { name: 'idx', maxCount: 1 },
]))
async uploadFile(
  @UploadedFiles() files: { file?: Express.Multer.File[], idx?: Express.Multer.File[] },
  @Body() body,
  @Request() req
) {
  const file = files.file ? files.file[0] : null;
  const idxFile = files.idx ? files.idx[0] : null;

  if (!file) {
    throw new BadRequestException('No .ld file uploaded');
  }
  // подальша логіка обробки
}
```

